

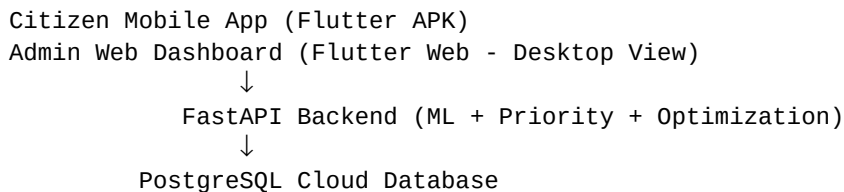
PREDICTIVE & PRIORITY-DRIVEN URBAN ISSUE MANAGEMENT SYSTEM

Complete Implementation & Deployment Plan

1. PROJECT OVERVIEW

This project is a full-stack AI-powered civic issue management system that enables citizens to report urban infrastructure problems such as road damage, waste overflow, and streetlight failures. The system automatically classifies issues using machine learning, removes duplicate reports, assigns dynamic priority scores, predicts future hotspots, and optimizes municipal resource allocation.

2. FINAL SYSTEM ARCHITECTURE



3. TECHNOLOGY STACK (100% FREE)

- Frontend: Flutter (Android APK + Web Dashboard)
- Backend: FastAPI (Python)
- Database: PostgreSQL (Supabase Cloud Hosting)
- ML Image Model: TensorFlow + MobileNetV2
- ML Text Model: TF-IDF + SVM (Scikit-learn)
- Forecasting: Prophet
- Optimization: PuLP (Linear Programming)
- Deployment Backend: Render (Free Tier)
- Web Hosting: Firebase Hosting
- ML Training: Google Colab (Free GPU)
- Version Control: GitHub
- IDE: Visual Studio Code

4. COMPLETE PROJECT STRUCTURE

```
urban-ai-system/
├── frontend_flutter/
│   ├── lib/
│   │   ├── main.dart
│   │   ├── citizen/
│   │   │   ├── screens/
│   │   │   ├── widgets/
│   │   │   └── services/
│   │   ├── admin/
│   │   │   ├── screens/
│   │   │   ├── widgets/
│   │   │   └── common/
│   │   │       ├── models/
│   │   │       └── constants.dart
│   │   └── pubspec.yaml
│   └── backend_fastapi/
│       ├── app/
│       │   ├── main.py
│       │   ├── database.py
│       │   ├── models/
│       │   ├── routes/
│       │   └── services/
│       │       ├── image_classifier.py
│       │       ├── text_classifier.py
│       │       ├── duplicate_checker.py
│       │       ├── priority_engine.py
│       │       ├── forecasting_service.py
│       │       └── optimizer.py
│       ├── utils/
│       └── requirements.txt
├── ml_training/
│   ├── image_model/
│   ├── text_model/
│   └── forecasting/
├── docs/
│   └── project_report.docx
└── README.md
```

5. CORE SYSTEM MODULES

- Citizen Module: Report issues, GPS capture, image upload, status tracking.
- Admin Module: View dashboard, priority list, assign crew, monitor SLA.

- Image Classification: MobileNetV2 model for detecting issue category.
- Text Classification: SVM model for severity and department classification.
- Duplicate Detection: Perceptual image hash + Geo distance comparison.
- Priority Engine: Weighted scoring based on severity, age, upvotes, risk.
- Forecasting Module: Prophet-based hotspot prediction.
- Optimization Module: Linear Programming crew allocation.

6. DEVELOPMENT ROADMAP (12 WEEKS)

- Weeks 1-2: Setup Flutter, FastAPI, PostgreSQL connection.
- Weeks 3-4: Train and integrate image classification model.
- Week 5: Train and integrate text classification model.
- Weeks 6-7: Implement duplicate detection and priority scoring.
- Week 8: Implement forecasting module.
- Week 9: Implement optimization engine.
- Weeks 10-11: Frontend integration and testing.
- Week 12: Cloud deployment and final testing.

7. DEPLOYMENT PLAN

Citizen App Deployment:

Run: `flutter build apk --release`

Install generated APK on Android devices.

Admin Dashboard Deployment:

Run: `flutter build web`

Deploy build/web folder to Firebase Hosting.

Backend Deployment:

Deploy FastAPI application to Render.

Database Deployment:

Use Supabase PostgreSQL cloud instance.

8. FINAL SYSTEM OUTPUT

The final system includes: - Android Mobile Application for citizens - Desktop/Web Admin Dashboard - Cloud-hosted backend server - AI-powered classification system - Forecasting heatmaps - Optimization-based crew allocation - Secure authentication and database management
This project demonstrates full-stack development, machine learning integration, predictive analytics, and cloud deployment in an industry-level architecture.